Backpropagation và Khởi tạo tham số trong Deep Learning

Chain rule, gradient flow và Parameter Initialization

Bùi Đức Quân – 2026

1 Giới thiệu

Huấn luyện mạng nơ-ron là bài toán tối ưu hoá tham số θ theo hàm mất mát $L(\theta)$. Hai câu hỏi nền tảng quyết định việc mạng có học được hay không: (1) gradient $\nabla_\theta L$ được tính thế nào khi mạng có hàng triệu tham số; và (2) tham số được khởi tạo ở giá trị nào. Báo cáo trả lời hai câu hỏi trên bằng giải thích toán học và thực nghiệm nhỏ trên Fashion-MNIST. Câu hỏi nghiên cứu là: *khởi tạo tham số ảnh hưởng thế nào đến lan truyền gradient và khả năng học của mạng sâu?*

Đóng góp chính: (i) trình bày backpropagation như ứng dụng có hệ thống của chain rule, khác với gradient descent; (ii) chỉ ra ba cơ chế thất bại ban đầu (symmetry, vanishing, exploding) trong cùng một kiến trúc; (iii) đo gradient và variance tại khởi tạo cho năm cách khởi tạo.

2 Nền tảng: gradient và chain rule

Với hàm nhiều biến $L(\theta)$, gradient là vector các đạo hàm riêng $\nabla_\theta L = [\partial L/\partial\theta_1, \ldots, \partial L/\partial\theta_n]^\top$. Gradient chỉ hướng tăng nhanh nhất của L. Gradient descent cập nhật

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta\, \nabla_\theta L, \tag{1}$$

với η là learning rate. Dấu trừ đưa tham số đi ngược hướng tăng của L.

Với một đồ thị tính toán $z = g(u)$, $L = f(z)$, chain rule đa biến cho $\partial L/\partial u = (\partial L/\partial z)(\partial z/\partial u)$. Trong mạng nhiều lớp, mỗi lớp là một ánh xạ với Jacobian riêng; gradient qua nhiều lớp là tích của các Jacobian này.

3 Backpropagation

Forward pass tính đầu ra và lưu các giá trị trung gian $h^{(l)}$. Backward pass nhận vector gradient $\delta^{(l)} = \partial L/\partial h^{(l)}$ từ lớp trên và tính

$$\delta^{(l-1)} = J_l^\top \delta^{(l)}, \qquad \frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (h^{(l-1)})^\top, \tag{2}$$

trong đó J_l là Jacobian của lớp l. Thực tế không lập ma trận J_l đầy đủ: backprop chỉ thực hiện tích vector–Jacobian (VJP). Autograd của PyTorch thực hiện VJP cho từng nút trong đồ thị.

Backpropagation chỉ *tính* gradient. Việc *cập nhật* tham số là vai trò của optimizer (gradient descent hoặc biến thể). Phân biệt này quan trọng khi đọc các thí nghiệm: nếu có vấn đề về gradient, nguyên nhân nằm ở lan truyền ngược, không ở bước cập nhật.

4 Vanishing và exploding gradient

Gradient ở lớp đầu là tích của nhiều Jacobian. Nếu mỗi Jacobian làm co norm của tín hiệu, gradient tiến về 0 (*vanishing*); nếu khuếch đại, gradient tăng theo cấp số nhân (*exploding*). Minh hoạ đơn giản: nếu mỗi lớp co norm theo hệ số 0,5, sau 20 lớp hệ số là $0{,}5^{20} \approx 9{,}5 \times 10^{-7}$. Trong thực nghiệm, cùng cơ chế này xuất hiện qua scale của weight khởi tạo.

Dạng hiện tượng phụ thuộc vào activation: với sigmoid và tanh, pre-activation lớn đẩy đạo hàm về 0 do bão hoà; với ReLU, đạo hàm bằng 0 ở miền âm và scale tín hiệu giảm xấp xỉ một nửa mỗi lớp về mặt second moment.

5 Khởi tạo tham số

5.1 Zero initialization: thất bại đối xứng

Nếu toàn bộ weight bằng 0, mọi neuron trong một lớp nhận cùng input, cùng weight, do đó có cùng pre-activation, cùng activation và cùng gradient. Sau mỗi bước cập nhật các neuron vẫn giống nhau. Lớp thoái hoá thành một neuron hiệu dụng; đây là *failure to break symmetry*. Vấn đề nằm ở weight; bias khởi tạo bằng 0 vẫn chấp nhận được vì đối xứng đã được tạo ra bởi weight.

5.2 Random initialization

Với $W \sim \mathcal{N}(0, \sigma^2)$:

- σ nhỏ (ví dụ 0,01) làm tín hiệu co dần qua các lớp, gây *vanishing* trong thiết lập thực nghiệm dưới đây;
- σ lớn (ví dụ 1,0) làm tín hiệu và gradient phình ra, gây *exploding* và loss trở thành NaN trong thiết lập này.

Kết luận này thuộc setup MLP với ReLU trên Fashion-MNIST đã thử; không khẳng định cho mọi kiến trúc.

5.3 Xavier/Glorot và He/Kaiming

Xavier (dạng Normal) chọn $\mathrm{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$. Giả định: activation gần tuyến tính quanh 0, hoặc đối xứng quanh 0 như tanh, và các weight độc lập. Mục tiêu là giúp duy trì variance của tín hiệu qua các lớp, không phải đảm bảo tuyệt đối.

Với ReLU, giả định trên không đúng: ReLU triệt tiêu miền âm, làm second moment $\mathbb{E}[a^2]$ giảm xấp xỉ một nửa mỗi lớp. He/Kaiming chọn $\mathrm{Var}(W) = 2/n_{\text{in}}$ để bù lại hệ số này. Lưu ý: đại lượng được điều chỉnh là $\mathbb{E}[a^2]$; variance chỉ bằng $\mathbb{E}[a^2]$ khi trung bình xấp xỉ 0, mà ReLU không zero-centered.

6 Thiết lập thực nghiệm

Thành phần	Giá trị
Kiến trúc	MLP $784 \to 128 \times 10 \to 10$, ReLU
Dữ liệu	Fashion-MNIST subset: 5.000 train / 1.000 val / 2.000 test
Chuẩn hoá	Mean/std tính trên tập train
Optimizer	SGD thuần, $\eta = 0{,}05$, batch 128, 15 epoch, không momentum, không scheduler
Loss	CrossEntropy
Bias	Khởi tạo 0 trong mọi cấu hình
Seed	42 (5 seed $\{42, \ldots, 46\}$ cho Xavier và He)
Biến duy nhất	Cách khởi tạo weight

Bảng 1: Thiết lập thực nghiệm cố định. Mọi cấu hình khác chỉ đổi cách khởi tạo weight.

Gradient RMS được đo bằng $\sqrt{\text{mean}(g^2)}$ trên batch đầu tiên, trước mọi cập nhật. Activation variance đo tương tự trên cùng batch.

7 Kết quả

7.1 Gradient và activation tại khởi tạo

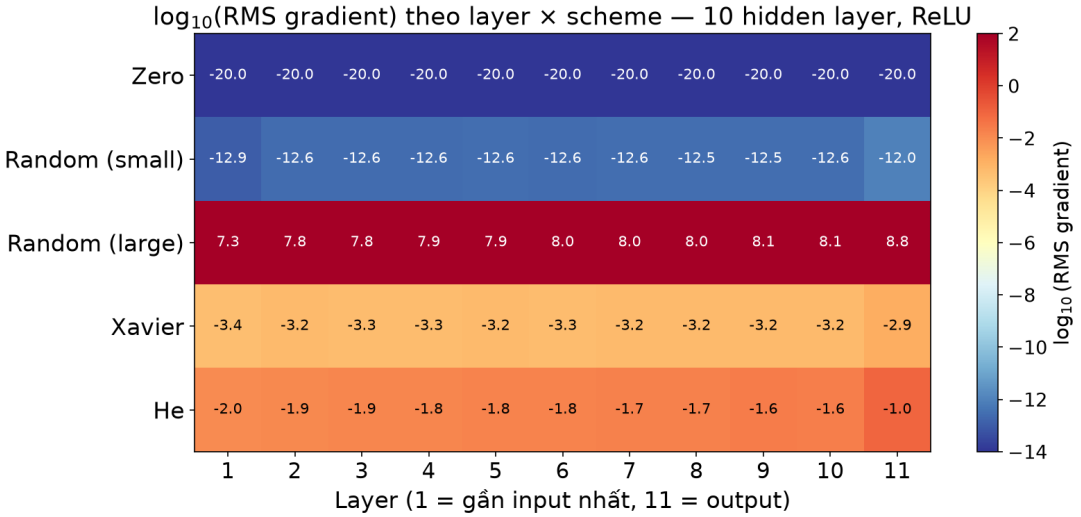
Bảng 2 tóm tắt gradient lớp 1 và kết quả test cuối cùng cho các cấu hình trong thiết lập chính (10 hidden layer, ReLU, seed 42).

Khởi tạo	Gradient RMS lớp 1 (khởi tạo)	Test accuracy	Hành vi
Zero	0	10,0%	Symmetry failur
Random nhỏ ($\sigma = 0{,}01$)	$1{,}1 \times 10^{-13}$	10,0%	Vanishing
Random lớn ($\sigma = 1{,}0$)	$1{,}9 \times 10^{7}$	10,0% (loss NaN)	Exploding → Na
Xavier (Normal)	$4{,}4 \times 10^{-4}$	68,2%	Không vanish/explode
He (Kaiming)	$1{,}1 \times 10^{-2}$	67,4%	Không vanish/explode

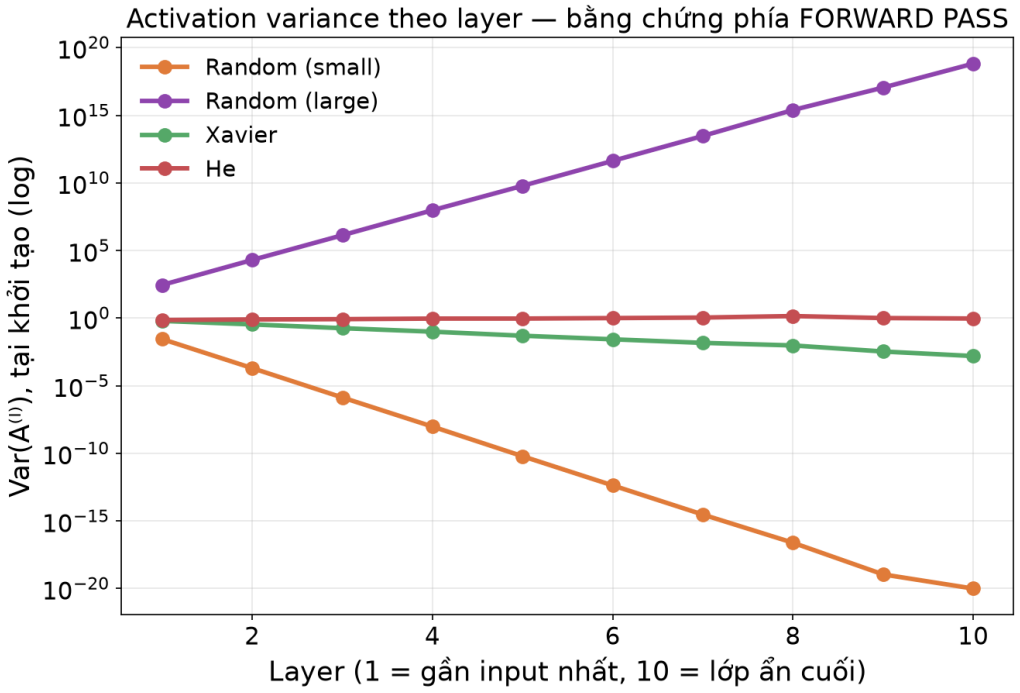
Bảng 2: Kết quả tại khởi tạo và test accuracy sau 15 epoch (seed 42).

Ba cơ chế thất bại được phân biệt rõ: Zero-init thất bại vì đối xứng (gradient đúng bằng 0 và các neuron không phân biệt được); Random nhỏ thất bại vì vanishing; Random lớn thất bại vì exploding, và NaN chỉ là triệu chứng của mất ổn định số học.

Hình 1 cho thấy gradient RMS theo lớp và cách khởi tạo: vùng xanh là gradient nhỏ (vanishing), vùng đỏ là gradient lớn (exploding). Hình 2 cho thấy forward pass: Random nhỏ làm variance co lại, Random lớn làm variance tăng mạnh, He gần như phẳng và Xavier vẫn giảm nhưng hẹp hơn Random nhỏ rất nhiều. Cả hai hình đo tại khởi tạo trên một batch, trước mọi cập nhật, không phải variance trung bình trong huấn luyện.

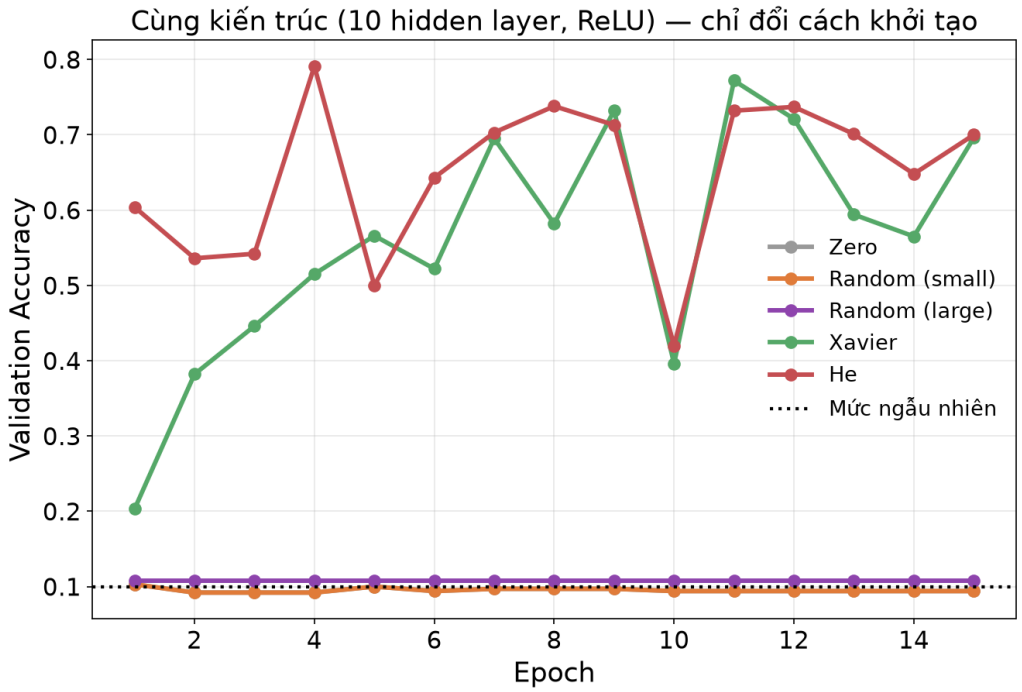


Hình 1: Gradient RMS (thang $\log_{10}$) theo lớp và cách khởi tạo, đo tại khởi tạo trên một batch.



Hình 2: Activation variance theo lớp tại khởi tạo (forward pass), một batch, trước mọi cập nhật.

7.2 Quỹ đạo huấn luyện



Hình 3: Validation accuracy theo epoch cho 5 cách khởi tạo cùng kiến trúc (10 hidden layer, ReLU).

Zero và Random nhỏ đứng ở mức 10%. Random lớn không hội tụ. Xavier và He đạt khoảng 67–68% test accuracy. Dao động của validation accuracy đáng kể vì dùng vanilla SGD với $\eta = 0{,}05$, không momentum và không scheduler, trên mạng 10 tầng; đây không phải tính chất riêng của Xavier hay He.

7.3 Xavier so với He

Trên 5 seed, test accuracy trung bình $\pm$ độ lệch chuẩn là $69{,}1\% \pm 4{,}7$pp cho Xavier và $70{,}5\% \pm 5{,}7$pp cho He. Khoảng mean $\pm$ 1 std chồng lấn đáng kể. Gradient RMS lớp 1 của He lớn hơn Xavier khoảng 35 lần (trung bình $1{,}5 \times 10^{-2}$ so với $4{,}2 \times 10^{-4}$), nhất quán trên mọi seed, nhưng gradient lớn hơn không đồng nghĩa với kết quả tốt hơn. Với số seed hiện tại, **chưa đủ cơ sở xếp hạng Xavier và He** trong thiết lập này.

7.4 Độ sâu



Hình 4: Gradient RMS lớp 1 tại khởi tạo theo độ sâu, cho Random, Xavier và He.

Khi độ sâu tăng, gradient của Random và Xavier suy giảm; He giữ scale tốt hơn trong thiết lập này. Ổn định gradient không đồng nghĩa với generalization tốt: ở độ sâu 12, He có train loss thấp nhưng test loss cao. Kết luận chỉ đúng trong kiến trúc MLP, ReLU, Fashion-MNIST và SGD đã thử; không suy ra rằng He ổn định ở mọi độ sâu.

8 Thảo luận: mạng hiện đại

Trong mạng rất sâu (trên khoảng 50 lớp), initialization thường không đủ một mình. Các kiến trúc hiện đại kết hợp với Residual connections (skip connections), cho gradient đường đi trực tiếp, và Normalization (LayerNorm, BatchNorm), chuẩn hoá tín hiệu ở mỗi lớp. Các kỹ thuật này được khuyến nghị dùng cùng với initialization để kiểm soát gradient flow; báo cáo không thực nghiệm chúng.

9 Hạn chế

- Dataset là subset nhỏ (5k/1k/2k) và chỉ một kiến trúc MLP với ReLU.
- Optimizer cố định (SGD, $\eta = 0{,}05$); không khảo sát tương tác với learning rate hay optimizer khác.
- Gradient và activation variance đo tại khởi tạo trên một batch; không phản ánh hành vi trong suốt huấn luyện.
- Xavier và He có 5 seed; Zero, Random nhỏ và Random lớn chỉ có 1 seed.

- Thí nghiệm độ sâu dùng số epoch ít hơn, số tuyệt đối có thể khác khi huấn luyện đủ lâu.

10 Kết luận

1. Backpropagation là chain rule áp dụng ngược qua đồ thị tính toán để tính gradient; gradient descent là quy tắc cập nhật dùng gradient đó.
2. Gradient ở lớp đầu là tích nhiều Jacobian, nên có thể vanish hoặc explode; khởi tạo quyết định scale ban đầu của tín hiệu và gradient.
3. Trong thiết lập đã thử, zero-init gây thất bại đối xứng, random nhỏ gây vanishing, random lớn gây exploding; Xavier và He tránh được các thất bại này.
4. Chưa đủ dữ liệu để xếp hạng Xavier và He; gradient ổn định không đảm bảo generalization tốt.

A Phụ lục: mã nguồn và tái lập

Mã nguồn đầy đủ nằm trong thư mục `src/` và `experiments/`. Các lệnh tái lập kết quả:

Listing 1: Lệnh tái lập

```
python experiments/run_manual_backprop.py
python experiments/run_gradient_check.py
python experiments/run_deep_demo.py
python experiments/run_multiseed.py
python experiments/run_depth_experiment_v2.py
```

Listing 2: Tính gradient RMS tại khởi tạo (trích, `experiments/run_deep_demo.py`)

```
loss = criterion(model(x), y)
loss.backward()
grad_rms = [p.grad.pow(2).mean().sqrt().item()
          for n, p in model.named_parameters() if "weight" in n]
```

Tài liệu

[1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[2] Y. LeCun, L. Bottou, G. B. Orr, K.-R. Müller. Efficient BackProp. In *Neural Networks: Tricks of the Trade*, 1998/2012.

[3] X. Glorot, Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. *AISTATS*, 2010.

[4] K. He, X. Zhang, S. Ren, J. Sun. Delving Deep into Rectifiers. *ICCV*, 2015. arXiv:1502.01852.

[5] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.

[6] H. Xiao, K. Rasul, R. Vollgraf. Fashion-MNIST. arXiv:1708.07747, 2017.

[7] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.